

How-To and Templates

Page Content

- [Generate SDK Documents](#)
 - [Step 1\) Create Documentation in Wiki](#)
 - [Step 2\) Generate Word Files from Wiki](#)
 - [Step 3\) Touch up Word Files](#)
 - [Step 4\) Generate PDF from Word Files](#)
 - [Step 5\) Create TXT Files](#)
- [Doxygen](#)
 - [ToDo: Explain How it Works](#)
 - [Browsable Documentation in DevNet](#)
- [TRENTOS Documentation](#)
 - [Handbook](#)
 - [v0.9](#)
 - [v1.0](#)
 - [seL4/CAMkES](#)

Sub-Pages

- [Document Style Guide](#)
- [API Template](#)
- [Component Template](#)
- [Interface Template](#)

Generate SDK Documents

We are currently creating the following files for our SDK documentation:

- [Getting Started \(PDF\)](#)
- [Handbook \(PDF\)](#)
- [Release Notes \(PDF\)](#)
- [Migration Guide \(PDF\)](#)
- [README.txt](#)
- [README_internal.txt](#)
- [errata.txt](#)

In general the pages shall follow a similar structure and style. Style guidelines and templates can be found in the subpages of this page.

Step 1) Create Documentation in Wiki

- The Getting Started and Handbook are generated from the corresponding and non-version-specific page structures below in [TRENTOS Documentation](#). After finalizing the pages for a release, the generated Word and PDF files are added to the respective subpages in [TRENTOS SDK Version](#).
- The version-specific files (Release Notes, Migration Guide) are stored in the respective subpages in [TRENTOS SDK Version](#) directly.
- The content of the version-specific files (README.txt, README_internal.txt and errata.txt) is stored in the respective subpages in [TRENTOS SDK Version](#) directly.

Step 2) Generate Word Files from Wiki

In this step for all PDF files at first a Word file is generated from a Wiki page and it's subpages. For that the Confluence Plugin "Scroll Exporter for Word" is used.

- Go to the page you want to export
- Click on the three dots in the top right corner of the browser window.
- Select "Export to Word"

Option	Setting
Export Template	TRENTOS Handbook Template v.5 Draft *
Export Scope	This Page and its Children
Details => Content => Enable Links to External Websites	Yes
Details => Content => Enable Links to Confluence	No
Details => Content => Export thumbnail images in full resolution	Yes

Details => Content => Merge single, first heading and page title	Yes
Details => Content => Allow Word macros	No (will be done later)
Details => Macros => Show "Table Of Content" macro output	No (but hidden via Scroll Ignore anyway)
Details => Macros => Show "Children" macro output	Yes (not used currently)
Details => Macros => Convert image gallery tables to lists	No
Details => Macros => Keep multi-column page layouts	No (not used currently)
Details => Captions => Figure Captions	Display after
Details => Captions => Table Captions	Display after
Details => Tables => Table Fitting	Auto fit to window

*) Steps to access and change the export templates (**admin rights required**):

- Click space tools in the lower left corner in wiki
- Select apps
- Select "Scroll Exporter for Word" in the navigation menu
- Click the three dots behind the last published template and select edit
- Download the Word template or upload a new Word template right away
- Make sure to click save in the upper right corner to submit the changes

Step 3) Touch up Word Files

Now the files are made pretty for final release. This includes (not complete):

- Modify text size and line break on Title Page
- Remove empty Lists (Lists of tables, List of figures) depends on concrete file
- Solve any missing / broken links
- Check page breaks and manually adjust if necessary (normally only necessary for very long code blocks / tables / images spanning multiple pages)
- Check for orphaned texts or empty pages (happens e.g. after long tables followed by automatic new page for Level 1 headings)
- Check for correct captions
 - currently code blocks are exported with caption "Table" initially
 - Solution:
 - select complete caption of first code block labeled as "Table"
 - Select Word option "Verweise => Beschriftung einfügen" (TBD english option)
 - Select "Code Block" caption type from drop down menu, Confirm
 - => magically all Code blocks are now correctly labelled
- Update all references in document (Select All, F9; several times)
- Remove non-necessary information from Word file (Datei => Auf Probleme prüfen; Remove private date, etc.)

Step 4) Generate PDF from Word Files

- Generate PDF files with PDF converter of Word
- Check the final result

Step 5) Create TXT Files

- Create README and errata text files with the content on the corresponding pages

Doxygen

- TRENTOS Doxygen Documentation is currently hosted on the hc-artefact server (https://hc-artefact/documentation/trentos_sdk/) in the [Development Network](#).
- It's automatically built by CI from the master and integration branches.
- For Doxygen see also [SW Team - Doxygen Guidelines](#).

ToDo: Explain How it Works

- the TRENTOS Teaching Material covers
 - API
 - CAMkES extensions
 - SDK build system
 - sample code
 - tests
- [SEOS tests](#) has a documentation build configuration which internally builds the ninja target "seos_crypto_doc"

```
src/build.sh DOC
```

Browsable Documentation in DevNet

- TRENTOS Libraries and Tests <http://seos-doc.devnet.hensoldtcyber>
Note: documentation gets built from the master and integration branches of [SEOS tests](#). Therefore in some circumstance the rebuild of it is triggered resulting in a temporary (usually not more than 5 mins) unavailability.

TRENTOS Documentation

Handbook

v0.9

see [TRENTOS Handbook](#)

v1.0

see [v1.0 - Planning](#)

seL4/CAmkES

- see also [CAmkES Notes](#) and [FAQ](#) for some general information about CAmkES
- When using TRENTOS component then the main CMakeLists.txt file must use

```
DeclareAndCreateCamkESSystem(<file>)
```

instead of

```
DeclareCAmkESRootserver(<file>)  
GenerateCAmkESRootServer()
```

Because additional definitions are passed to DeclareCAmkESRootserver() by the SDK then, which are required for the build to work

- When the CAmkES tool runs, the preprocessor will be invoked with CAMKES_TOOL_PROCESSING, so header files can use this to hide thing from CAmkES, e.g. C constructs the are not supported by the parser
- the system project base folder and the SDK component base directory is added to the include path list
- CAmkES component macros
 - components/ChanMux
 - CMakeLists.txt defines the CMake macro

```
DeclareCAmkESComponent_ChanMux(<name> <config_file> <system_config_lib>)
```

for example

```
DeclareCAmkESComponent_ChanMux(  
    ChanMux                # Name  
    components/ChanMux/ChanMux.c # defines const ChanMux_config_t cfgChanMux  
    system_config           # provides include paths for the config file above  
)
```

- ChanMux.camkes can be included to define a CanMux component, usage is

```

#define CHANMUX_COMPONENT_NAME  ChanMux
#define CHANMUX_UPPER_INTERFACES "components/ChanMux/ChanMux_upper_interface.camkes"
#include "ChanMux/ChanMux.camkes"

...

assembly {
    composition {

        ...

        DECLARE_AND_CONNECT_CHANMUX_INSTANCE(
            ChanMux, chanmux,
            <uart_rpc_handler_interface>, <dataport_used_for_rpc_call_to_uart>
            <uart_rpc_caller_interface>)

        ...
    }
}

```

this can be done multiple times also. The custom file components/ChanMux/ChanMux_upper_interface.camkes contains the ChanMux client specific interfaces, e.g.

```

// The file components/ChanMux/ChanMux.camkes from the generic ChanMux CAMkes
// component will include this

//-----
// upper interface to Ethernet NIC driver
dataport      Buf          nic_port_ctrl;
dataport      Buf          nic_port_data_read;
dataport      Buf          nic_port_data_write;
emits         EventDataAvailable  nic_event_hasData;

```

- components/NIC_ChanMux

- CMakeLists.txt defines the CMake macro

```
DeclareCAMkesComponent_NIC_ChanMux(<name> <channel_ID_ctrl> <channel_ID_data>)
```

for example

```

DeclareCAMkesComponent_NIC_ChanMux(
    NwDriver
    CHANMUX_CHANNEL_NIC_CTRL
    CHANMUX_CHANNEL_NIC_DATA
)

```

ToDo: Clarify why this works and CHANMUX_CHANNEL_NIC_CTRL can be resolved against system_config.h, even it this is not passed anywhere explicitly?

- NIC_ChanMux.camkes defines the macro to create a NIC instance in a CAMkes file

```
DECLARE_COMPONENT_NIC_ChanMux(component_name)
```

which is used in the system project CAMkes file lie e.g.

```

#include "NIC_ChanMux/NIC_ChanMux.camkes"
DECLARE_COMPONENT_NIC_ChanMux(my_nic_1)
DECLARE_COMPONENT_NIC_ChanMux(my_nic_2)

```